

AssetHub API

Frontend Integration and Endpoint Reference

Generated 2026-09-22

Base URL: `http://localhost:3000/api` or `https://assethub-backend.vercel.app/api`

Frontend integration

Production API: `https://assethub-backend.vercel.app/api`.

The changes in this checkout require deployment and the maintenance-trigger migration before production exposes the new behavior.

Login and tokens

```
const baseUrl = 'https://assethub-backend.vercel.app/api';
const response = await fetch(`${baseUrl}/auth/login`, {
  method: 'POST', headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email: 'admin@assethub.local', password: 'Admin@123' })
});
const result = await response.json();
if (!response.ok) throw new Error(result.error.message);
let accessToken = result.data.accessToken;
const assets = await fetch(`${baseUrl}/assets?page=1&limit=10`, {
  headers: { Authorization: `Bearer ${accessToken}` }
});
```

Keep the access token in application memory and clear it on logout. Avoid persistent localStorage for bearer tokens. Use the existing `/auth/refresh` API with `{ "refreshToken": "..." }` to rotate tokens, or ask the user to log in again on 401. Never put tokens in URLs. Handle 403 separately from 401.

Responses use `{ "data": { ... } }`; lists also contain `meta: { page, limit, total, pages }`. Errors use `{ "error": { "code", "message" } }`. Decimal money values are returned as strings; send nonnegative JSON numbers in requests. Timestamps are ISO strings.

Endpoints

All paths below are relative to the base URL. Read requests require authentication; invoice reads also require `procurement:manage`. User location scope is enforced on inventory and operational records. ADMIN bypasses permission checks.

Resource	Methods	Write permission	List filters beyond page/limit
<code>/locations</code>	GET, POST; GET/PUT/DELETE <code>{id}</code>	<code>location:manage</code>	search, parentId, tree
<code>/assets</code>	GET, POST; GET/PUT/DELETE <code>{id}</code>	<code>asset:write</code> ; delete: <code>asset:delete</code>	search, status, condition, categoryId, locationId
<code>/suppliers</code>	GET, POST; GET/PUT/DELETE <code>{id}</code>	<code>procurement:manage</code>	search (name/email/phone)
<code>/purchase-orders</code>	GET, POST; GET/PUT/DELETE <code>{id}</code>	<code>procurement:manage</code>	status
<code>/invoices</code>	GET, POST; GET/PUT/DELETE <code>{id}</code>	<code>procurement:manage</code>	status
<code>/warranties</code>	GET, POST; GET/PUT/DELETE <code>{id}</code>	<code>procurement:manage</code>	assetId

Resource	Methods	Write permission	List filters beyond page/limit
/custody-assignments	GET, POST; GET/PUT /{id}	custody:manage	assetId, active=true/false
/transfers	GET, POST; GET /{id}	transfer:manage	assetId
/maintenance-templates	GET, POST; GET/PUT/DELETE /{id}	maintenance:manage	none
/work-orders	GET, POST; GET/PUT /{id}	workorder:manage	status, priority, assetId, assignedToUserId, scheduledFrom, scheduledTo
/service-events	GET, POST	workorder:manage	workOrderId

Additional reads: /locations/tree, /assets/{id}/history, /warranties/expiring?days=30, /work-orders/due. Completion: PUT /work-orders/{id}/complete with workorder:manage.

See [API reference](api-reference.md) for every request body, success example, permission and error response; [OpenAPI](openapi.json) and [Postman](postman.json) contain the same endpoint contracts. PUT is a partial update. Omit optional links rather than sending placeholder IDs. Service creation also accepts optional eventDate, technician, and cost; supplied cost must equal labor plus parts.

Values and rules

- Location types, case-insensitive: campus, building, college, floor, room, office, LOCATION (legacy generic type). Named levels follow campus → building → college → floor → room/office; intermediate levels can be omitted. Cycles and reversed levels are rejected. Generic locations retain legacy flexibility.
- Asset status and condition remain free-form strings for existing data compatibility. Seed examples include ACTIVE/RETIRED and GOOD; filters match stored values exactly. Tags and non-null serial numbers are unique. Use null for unknown serial numbers.
- Purchase status: DRAFT, APPROVED, ORDERED, RECEIVED, CANCELLED. orderNumber is the reference number; orderDate is the stored purchase date.
- Invoice status: UNPAID, PAID, OVERDUE, CANCELLED. Link a supplier or purchase order; when both are supplied, suppliers must match. Due date cannot precede issue date.
- Warranty end date must follow start date. Historical expired warranties are allowed. Compare endDate to the current time for expiry; /warranties/expiring returns future expirations within days.
- Template triggerType: TIME_BASED or MANUAL. TIME_BASED requires positive integer frequencyDays. Checklist is tasks (string or array of strings); arrays are stored and returned as JSON text. Parse that text when rendering checkboxes. Description and categoryId are optional.
- Priority: LOW, MEDIUM, HIGH, CRITICAL. Status: OPEN, ASSIGNED, IN_PROGRESS, COMPLETED, CANCELLED. assignedToUserId identifies a technician/responsible user. The existing schema does not restrict assignment to a particular role. dueDate is the scheduling field; scheduledFrom/scheduledTo filter it inclusively.
- Service outcome is a nonempty free-form string, default completed. Labor, parts, total and downtimeHours cannot be negative. Total is stored in cost using decimal arithmetic.
- POST returns 201; GET/PUT/DELETE return 200. Invalid rules return 400, missing links 404, duplicates/protected deletes/terminal edits/concurrent writes 409. On a concurrent-write 409, refresh the record before retrying.

Workflow order

- Create category (existing `/categories` API), location, then asset. Asset PUT changes descriptive fields; use the workflows below for location and custodian changes.
- Transfer: POST `/transfers` with `{ "assetId": "...", "toLocationId": "...", "reason": "Move to lab" }`. The API derives source location and actor and atomically updates inventory. Read the asset and its history afterward. Same-location transfers fail. Transfers preserve custody; their history cannot be edited/deleted.
- Custody: POST `/custody-assignments` with `{ "assetId": "...", "userId": "...", "notes": "Issued" }`. Duplicate active assignments return 409. Release using PUT `/id` with `{ "returnedAt": "<current ISO timestamp>", "notes": "Returned" }`, then assign again if needed. Actor is recorded in the atomic audit log. The schema supports users, not responsible units. Historical assignments cannot be deleted.
- Procurement: create supplier → purchase order → invoice; link warranty to asset. An invoiced purchase order cannot switch suppliers or be deleted; referenced suppliers cannot be deleted.
- Maintenance: create template → create OPEN work order → PUT status ASSIGNED (requires `assignedToUserId`) → IN_PROGRESS → PUT `/complete` with `{ "laborCost": 100, "partsCost": 50, "downtimeHours": 2, "outcome": "Fixed" }`. Completion atomically records service, completion date and audit. OPEN may also move directly to IN_PROGRESS. Any nonterminal status may move to CANCELLED. COMPLETED and CANCELLED are terminal. Generic PUT status COMPLETED also creates a zero-cost service event; use `/complete` to include costs.
- Add additional service with POST `/service-events` linked to the work order. Cancelled work orders reject service. Work-order deletion is blocked to preserve history. Templates with work orders cannot be deleted.

Locations with references cannot be deleted. Assets with procurement, maintenance, custody, transfer or other related history cannot be deleted; use the existing retirement workflow. Audit records survive deletion of otherwise unreferenced assets.

Local verification

Use a disposable PostgreSQL database whose name contains `test`. Set `TEST_DATABASE_URL` locally; never use the production connection. Each integration suite creates a random isolated schema and drops only that schema afterward. The test role needs schema creation/drop privileges. The acceptance seed runs only inside its isolated schema.

```
Set-Location 'D:\2nd Term Subjects\assethub-backend_lastttt\assethub-backend'
$env:TEST_DATABASE_URL = Read-Host 'Enter the real connection URL for your disposable PostgreSQL test database'
npx.cmd prisma validate
npx.cmd prisma generate
npm.cmd test
```

`.cmd` avoids the local PowerShell script execution-policy restriction and runs the same npm/npx commands. Alternatively put `TEST_DATABASE_URL` in an untracked `.env.test`.

For API workflow checks, start a local server configured against a disposable PostgreSQL database, then run:

```
powershell -ExecutionPolicy Bypass -File .\docs\test-workflows.ps1 -BaseUrl 'http://localhost:3000/api'
```

The script creates records and leaves their history for inspection. It deliberately refuses the production hostname. The additive migration `20260922_maintenance_trigger` must be applied with `npx.cmd prisma migrate deploy` as part of the normal deployment process. No production migration was run for this task.

Troubleshooting test startup

P1000 with user TEST_USER means the documentation placeholders were used literally. They are not an account created by AssetHub. Obtain real credentials for a separate PostgreSQL database whose name contains test (for example assethub_test). Do not use the production Neon database. Prisma validate/generate do not prove that database credentials work.

After setting a real TEST_DATABASE_URL, run npm.cmd test. Automated tests start Express through Supertest and do not need port 3000.

For manual workflows, run npm.cmd run test:server in the same terminal with TEST_DATABASE_URL set. This command creates a random isolated schema, prepares demo accounts in that schema, and starts Express at http://localhost:3000/api. Wait for the ready message, then run docs/test-workflows.ps1 from a second terminal. Stop with Ctrl+C to clean up the temporary schema. A forced process kill can leave that isolated schema behind. Production is never used as a fallback.

AssetHub module API reference

All routes below use `/api`. Send `Authorization: Bearer <accessToken>`. `Authenticated` means any authenticated user, subject to existing location scopes. ADMIN bypasses permission checks. PUT bodies are partial unless noted. Dates accept ISO strings. Monetary input is a JSON number; Prisma Decimal output is a string.

Lists accept integer `page` (default 1) and `limit` (default 20, maximum 100).

Common errors

- `400`: Invalid body, pagination, hierarchy, dates, costs, or lifecycle transition.
- `401`: Missing, expired or invalid bearer token.
- `403`: Insufficient permission or location outside user scope.
- `404`: Resource or linked record not found.
- `409`: Duplicate identifier, active custody, protected history, referenced deletion, terminal work order, or concurrent write conflict.
- `500`: Unexpected server error.

Error envelope: `{"error":{"code":"VALIDATION_ERROR","message":"..."}}`. Creation returns 201; reads, updates and deletions return 200.

GET /locations

Permission: `Authenticated`. Queries: `page`, `limit`, `search`, `parentId`, `tree`.

Request body: none.

Success `200`:

```
{
  "data": [
    {
      "id": "example-id",
      "name": "Room 101",
      "code": "ROOM-101",
      "type": "room",
      "parentId": "{{parentLocationId}}",
      "address": "Main campus",
      "description": "Lab"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "pages": 1
  }
}
```

Errors: `400`, `401`, `403`, `404`, `409`, `500` as defined above.

GET /locations/{id}

Permission: `Authenticated`. Queries: none.

Request body: none.

Success `200`:

```
{
  "data": {
    "id": "example-id",
    "name": "Room 101",
    "code": "ROOM-101",
    "type": "room",
    "parentId": "{{parentLocationId}}",
    "address": "Main campus",
    "description": "Lab"
  }
}
```

Errors: `400`, `401`, `403`, `404`, `409`, `500` as defined above.

POST /locations

Permission: `location:manage`. Queries: none.

Request (required: name, code):

```
{
  "name": "Room 101",
  "code": "ROOM-101",
  "type": "room",
  "parentId": "{{parentLocationId}}",
  "address": "Main campus",
  "description": "Lab"
}
```

Success `201`:

```
{
  "data": {
    "id": "example-id",
    "name": "Room 101",
    "code": "ROOM-101",
    "type": "room",
    "parentId": "{{parentLocationId}}",
    "address": "Main campus",
    "description": "Lab"
  }
}
```

Errors: `400`, `401`, `403`, `404`, `409`, `500` as defined above.

PUT /locations/{id}

Permission: `location:manage`. Queries: none.

Request (all fields optional):

```
{
  "name": "Room 101",
  "code": "ROOM-101",
  "type": "room",
  "parentId": "{{parentLocationId}}",
  "address": "Main campus",
  "description": "Lab"
}
```

Success 200:

```
{
  "data": {
    "id": "example-id",
    "name": "Room 101",
    "code": "ROOM-101",
    "type": "room",
    "parentId": "{{parentLocationId}}",
    "address": "Main campus",
    "description": "Lab"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

DELETE /locations/{id}

Permission: location:manage. Queries: none.

Request body: none.

Success 200:

```
{
  "data": {
    "deleted": true
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /assets

Permission: Authenticated. Queries: page, limit, search, status, condition, categoryId, locationId.

Request body: none.

Success 200:

```
{
  "data": [
    {
      "id": "example-id",
      "name": "Laptop",
      "assetTag": "LAP-001",
      "serialNumber": "SN-001",

```

```

        "categoryId": "{{categoryId}}",
        "locationId": "{{locationId}}",
        "status": "ACTIVE",
        "condition": "GOOD",
        "purchaseDate": "2026-01-01",
        "purchaseCost": "1000",
        "usefulLifeYears": 5,
        "value": "1000",
        "model": "Model A",
        "metadata": "{\\"department\\":\\"IT\\"}"
    }
],
"meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "pages": 1
}
}

```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /assets/{id}

Permission: `Authenticated`. Queries: none.

Request body: none.

Success 200:

```

{
  "data": {
    "id": "example-id",
    "name": "Laptop",
    "assetTag": "LAP-001",
    "serialNumber": "SN-001",
    "categoryId": "{{categoryId}}",
    "locationId": "{{locationId}}",
    "status": "ACTIVE",
    "condition": "GOOD",
    "purchaseDate": "2026-01-01",
    "purchaseCost": "1000",
    "usefulLifeYears": 5,
    "value": "1000",
    "model": "Model A",
    "metadata": "{\\"department\\":\\"IT\\"}"
  }
}

```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

POST /assets

Permission: `asset:write`. Queries: none.

Request (required: name, assetTag, categoryId, locationId):

```

{
  "name": "Laptop",

```

```
"assetTag": "LAP-001",
"serialNumber": "SN-001",
"categoryId": "{{categoryId}}",
"locationId": "{{locationId}}",
"status": "ACTIVE",
"condition": "GOOD",
"purchaseDate": "2026-01-01",
"purchaseCost": 1000,
"usefulLifeYears": 5,
"value": 1000,
"model": "Model A",
"metadata": {
  "department": "IT"
}
}
```

Success 201:

```
{
  "data": {
    "id": "example-id",
    "name": "Laptop",
    "assetTag": "LAP-001",
    "serialNumber": "SN-001",
    "categoryId": "{{categoryId}}",
    "locationId": "{{locationId}}",
    "status": "ACTIVE",
    "condition": "GOOD",
    "purchaseDate": "2026-01-01",
    "purchaseCost": "1000",
    "usefulLifeYears": 5,
    "value": "1000",
    "model": "Model A",
    "metadata": "{ \"department\": \"IT\" }"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

PUT /assets/{id}

Permission: `asset:write`. Queries: none.

Request (all fields optional):

```
{
  "name": "Laptop",
  "assetTag": "LAP-001",
  "serialNumber": "SN-001",
  "categoryId": "{{categoryId}}",
  "locationId": "{{locationId}}",
  "status": "ACTIVE",
  "condition": "GOOD",
  "purchaseDate": "2026-01-01",
  "purchaseCost": 1000,
  "usefulLifeYears": 5,
  "value": 1000,
  "model": "Model A",
  "metadata": {
    "department": "IT"
  }
}
```

```
}  
}
```

Success 200:

```
{  
  "data": {  
    "id": "example-id",  
    "name": "Laptop",  
    "assetTag": "LAP-001",  
    "serialNumber": "SN-001",  
    "categoryId": "{{categoryId}}",  
    "locationId": "{{locationId}}",  
    "status": "ACTIVE",  
    "condition": "GOOD",  
    "purchaseDate": "2026-01-01",  
    "purchaseCost": "1000",  
    "usefulLifeYears": 5,  
    "value": "1000",  
    "model": "Model A",  
    "metadata": "{\"department\":\"IT\"}"  
  }  
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

DELETE /assets/{id}

Permission: `asset:delete`. Queries: none.

Request body: none.

Success 200:

```
{  
  "data": {  
    "deleted": true  
  }  
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /suppliers

Permission: `Authenticated`. Queries: page, limit, search.

Request body: none.

Success 200:

```
{  
  "data": [  
    {  
      "id": "example-id",  
      "name": "Example Supplier",  
      "email": "sales@example.com",  
      "phone": "+201000000000",  
      "address": "Cairo"  
    }  
  ]  
}
```

```
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "pages": 1
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /suppliers/{id}

Permission: `Authenticated`. Queries: none.

Request body: none.

Success 200:

```
{
  "data": {
    "id": "example-id",
    "name": "Example Supplier",
    "email": "sales@example.com",
    "phone": "+201000000000",
    "address": "Cairo"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

POST /suppliers

Permission: `procurement:manage`. Queries: none.

Request (required: name):

```
{
  "name": "Example Supplier",
  "email": "sales@example.com",
  "phone": "+201000000000",
  "address": "Cairo"
}
```

Success 201:

```
{
  "data": {
    "id": "example-id",
    "name": "Example Supplier",
    "email": "sales@example.com",
    "phone": "+201000000000",
    "address": "Cairo"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

PUT /suppliers/{id}

Permission: `procurement:manage` . Queries: none.

Request (all fields optional):

```
{
  "name": "Example Supplier",
  "email": "sales@example.com",
  "phone": "+201000000000",
  "address": "Cairo"
}
```

Success 200:

```
{
  "data": {
    "id": "example-id",
    "name": "Example Supplier",
    "email": "sales@example.com",
    "phone": "+201000000000",
    "address": "Cairo"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

DELETE /suppliers/{id}

Permission: `procurement:manage` . Queries: none.

Request body: none.

Success 200:

```
{
  "data": {
    "deleted": true
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /purchase-orders

Permission: `Authenticated` . Queries: page, limit, status.

Request body: none.

Success 200:

```
{
  "data": [
    {
      "id": "example-id",
      "orderNumber": "PO-001",
      "supplierId": "{{supplierId}}",

```

```
    "assetId": "{{assetId}}",
    "status": "DRAFT",
    "orderDate": "2026-01-01",
    "totalAmount": "1000",
    "metadata": "{}"
  }
],
"meta": {
  "page": 1,
  "limit": 20,
  "total": 1,
  "pages": 1
}
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /purchase-orders/{id}

Permission: `Authenticated`. Queries: none.

Request body: none.

Success 200:

```
{
  "data": {
    "id": "example-id",
    "orderNumber": "PO-001",
    "supplierId": "{{supplierId}}",
    "assetId": "{{assetId}}",
    "status": "DRAFT",
    "orderDate": "2026-01-01",
    "totalAmount": "1000",
    "metadata": "{}"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

POST /purchase-orders

Permission: `procurement:manage`. Queries: none.

Request (required: orderNumber, supplierId):

```
{
  "orderNumber": "PO-001",
  "supplierId": "{{supplierId}}",
  "assetId": "{{assetId}}",
  "status": "DRAFT",
  "orderDate": "2026-01-01",
  "totalAmount": 1000,
  "metadata": {}
}
```

Success 201:

```
{
  "data": {
    "id": "example-id",
    "orderNumber": "PO-001",
    "supplierId": "{{supplierId}}",
    "assetId": "{{assetId}}",
    "status": "DRAFT",
    "orderDate": "2026-01-01",
    "totalAmount": "1000",
    "metadata": "{}"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

PUT /purchase-orders/{id}

Permission: `procurement:manage`. Queries: none.

Request (all fields optional):

```
{
  "orderNumber": "PO-001",
  "supplierId": "{{supplierId}}",
  "assetId": "{{assetId}}",
  "status": "DRAFT",
  "orderDate": "2026-01-01",
  "totalAmount": 1000,
  "metadata": {}
}
```

Success 200:

```
{
  "data": {
    "id": "example-id",
    "orderNumber": "PO-001",
    "supplierId": "{{supplierId}}",
    "assetId": "{{assetId}}",
    "status": "DRAFT",
    "orderDate": "2026-01-01",
    "totalAmount": "1000",
    "metadata": "{}"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

DELETE /purchase-orders/{id}

Permission: `procurement:manage`. Queries: none.

Request body: none.

Success 200:

```
{
  "data": {
```

```
    "deleted": true
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /invoices

Permission: `procurement:manage`. Queries: page, limit, status.

Request body: none.

Success 200:

```
{
  "data": [
    {
      "id": "example-id",
      "invoiceNumber": "INV-001",
      "purchaseOrderId": "{{purchaseOrderId}}",
      "supplierId": "{{supplierId}}",
      "amount": "1000",
      "issueDate": "2026-01-01",
      "dueDate": "2026-02-01",
      "status": "UNPAID",
      "metadata": "{}"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "pages": 1
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /invoices/{id}

Permission: `procurement:manage`. Queries: none.

Request body: none.

Success 200:

```
{
  "data": {
    "id": "example-id",
    "invoiceNumber": "INV-001",
    "purchaseOrderId": "{{purchaseOrderId}}",
    "supplierId": "{{supplierId}}",
    "amount": "1000",
    "issueDate": "2026-01-01",
    "dueDate": "2026-02-01",
    "status": "UNPAID",
    "metadata": "{}"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

POST /invoices

Permission: procurement:manage . Queries: none.

Request (required: invoiceNumber, amount, issueDate):

```
{
  "invoiceNumber": "INV-001",
  "purchaseOrderId": "{{purchaseOrderId}}",
  "supplierId": "{{supplierId}}",
  "amount": 1000,
  "issueDate": "2026-01-01",
  "dueDate": "2026-02-01",
  "status": "UNPAID",
  "metadata": {}
}
```

Success 201:

```
{
  "data": {
    "id": "example-id",
    "invoiceNumber": "INV-001",
    "purchaseOrderId": "{{purchaseOrderId}}",
    "supplierId": "{{supplierId}}",
    "amount": "1000",
    "issueDate": "2026-01-01",
    "dueDate": "2026-02-01",
    "status": "UNPAID",
    "metadata": "{}"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

PUT /invoices/{id}

Permission: procurement:manage . Queries: none.

Request (all fields optional):

```
{
  "invoiceNumber": "INV-001",
  "purchaseOrderId": "{{purchaseOrderId}}",
  "supplierId": "{{supplierId}}",
  "amount": 1000,
  "issueDate": "2026-01-01",
  "dueDate": "2026-02-01",
  "status": "UNPAID",
  "metadata": {}
}
```

Success 200:

```
{
  "data": {
    "id": "example-id",
    "invoiceNumber": "INV-001",
    "purchaseOrderId": "{{purchaseOrderId}}",
    "supplierId": "{{supplierId}}",
    "amount": "1000",
    "issueDate": "2026-01-01",
    "dueDate": "2026-02-01",
    "status": "UNPAID",
    "metadata": "{}"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

DELETE /invoices/{id}

Permission: `procurement:manage`. Queries: none.

Request body: none.

Success 200:

```
{
  "data": {
    "deleted": true
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /warranties

Permission: `Authenticated`. Queries: page, limit, assetId.

Request body: none.

Success 200:

```
{
  "data": [
    {
      "id": "example-id",
      "assetId": "{{assetId}}",
      "provider": "Manufacturer",
      "startDate": "2026-01-01",
      "endDate": "2027-01-01",
      "terms": "Parts and labor"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "pages": 1
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /warranties/{id}

Permission: `Authenticated`. Queries: none.

Request body: none.

Success 200:

```
{
  "data": {
    "id": "example-id",
    "assetId": "{{assetId}}",
    "provider": "Manufacturer",
    "startDate": "2026-01-01",
    "endDate": "2027-01-01",
    "terms": "Parts and labor"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

POST /warranties

Permission: `procurement:manage`. Queries: none.

Request (required: `assetId`, `startDate`, `endDate`):

```
{
  "assetId": "{{assetId}}",
  "provider": "Manufacturer",
  "startDate": "2026-01-01",
  "endDate": "2027-01-01",
  "terms": "Parts and labor"
}
```

Success 201:

```
{
  "data": {
    "id": "example-id",
    "assetId": "{{assetId}}",
    "provider": "Manufacturer",
    "startDate": "2026-01-01",
    "endDate": "2027-01-01",
    "terms": "Parts and labor"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

PUT /warranties/{id}

Permission: `procurement:manage`. Queries: none.

Request (all fields optional):

```
{
  "assetId": "{{assetId}}",
  "provider": "Manufacturer",
  "startDate": "2026-01-01",
  "endDate": "2027-01-01",
  "terms": "Parts and labor"
}
```

Success **200**:

```
{
  "data": {
    "id": "example-id",
    "assetId": "{{assetId}}",
    "provider": "Manufacturer",
    "startDate": "2026-01-01",
    "endDate": "2027-01-01",
    "terms": "Parts and labor"
  }
}
```

Errors: **400**, **401**, **403**, **404**, **409**, **500** as defined above.

DELETE /warranties/{id}

Permission: **procurement:manage**. Queries: none.

Request body: none.

Success **200**:

```
{
  "data": {
    "deleted": true
  }
}
```

Errors: **400**, **401**, **403**, **404**, **409**, **500** as defined above.

GET /custody-assignments

Permission: **Authenticated**. Queries: page, limit, assetId, active.

Request body: none.

Success **200**:

```
{
  "data": [
    {
      "id": "example-id",
      "assetId": "{{assetId}}",
      "userId": "{{userId}}",
      "assignedAt": "2026-01-01T00:00:00Z",
      "notes": "Issued to user"
    }
  ],
  "meta": {
```

```
    "page": 1,  
    "limit": 20,  
    "total": 1,  
    "pages": 1  
  }  
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /custody-assignments/{id}

Permission: `Authenticated`. Queries: none.

Request body: none.

Success 200:

```
{  
  "data": {  
    "id": "example-id",  
    "assetId": "{{assetId}}",  
    "userId": "{{userId}}",  
    "assignedAt": "2026-01-01T00:00:00Z",  
    "notes": "Issued to user"  
  }  
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

POST /custody-assignments

Permission: `custody:manage`. Queries: none.

Request (required: `assetId`, `userId`):

```
{  
  "assetId": "{{assetId}}",  
  "userId": "{{userId}}",  
  "assignedAt": "2026-01-01T00:00:00Z",  
  "notes": "Issued to user"  
}
```

Success 201:

```
{  
  "data": {  
    "id": "example-id",  
    "assetId": "{{assetId}}",  
    "userId": "{{userId}}",  
    "assignedAt": "2026-01-01T00:00:00Z",  
    "notes": "Issued to user"  
  }  
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

PUT /custody-assignments/{id}

Permission: `custody:manage`. Queries: none.

Request (required: returnedAt):

```
{
  "returnedAt": "2026-09-22T09:00:00Z",
  "notes": "Returned"
}
```

Success `200`:

```
{
  "data": {
    "id": "example-id",
    "assetId": "{{assetId}}",
    "userId": "{{userId}}",
    "assignedAt": "2026-01-01T00:00:00Z",
    "notes": "Issued to user"
  }
}
```

Errors: `400`, `401`, `403`, `404`, `409`, `500` as defined above.

GET /transfers

Permission: `Authenticated`. Queries: page, limit, assetId.

Request body: none.

Success `200`:

```
{
  "data": [
    {
      "id": "example-id",
      "assetId": "{{assetId}}",
      "toLocationId": "{{destinationLocationId}}",
      "reason": "Move to lab"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "pages": 1
  }
}
```

Errors: `400`, `401`, `403`, `404`, `409`, `500` as defined above.

GET /transfers/{id}

Permission: `Authenticated`. Queries: none.

Request body: none.

Success 200:

```
{
  "data": {
    "id": "example-id",
    "assetId": "{{assetId}}",
    "toLocationId": "{{destinationLocationId}}",
    "reason": "Move to lab"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

POST /transfers

Permission: transfer:manage. Queries: none.

Request (required: assetId, toLocationId):

```
{
  "assetId": "{{assetId}}",
  "toLocationId": "{{destinationLocationId}}",
  "reason": "Move to lab"
}
```

Success 201:

```
{
  "data": {
    "id": "example-id",
    "assetId": "{{assetId}}",
    "toLocationId": "{{destinationLocationId}}",
    "reason": "Move to lab"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /maintenance-templates

Permission: Authenticated. Queries: page, limit.

Request body: none.

Success 200:

```
{
  "data": [
    {
      "id": "example-id",
      "name": "Monthly inspection",
      "categoryId": "{{categoryId}}",
      "triggerType": "TIME_BASED",
      "frequencyDays": 30,
      "tasks": "[\"Inspect cables\", \"Check operation\"]",
      "description": "Monthly checks"
    }
  ],
}
```

```
"meta": {
  "page": 1,
  "limit": 20,
  "total": 1,
  "pages": 1
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /maintenance-templates/{id}

Permission: `Authenticated`. Queries: none.

Request body: none.

Success 200:

```
{
  "data": {
    "id": "example-id",
    "name": "Monthly inspection",
    "categoryId": "{{categoryId}}",
    "triggerType": "TIME_BASED",
    "frequencyDays": 30,
    "tasks": ["Inspect cables", "Check operation"],
    "description": "Monthly checks"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

POST /maintenance-templates

Permission: `maintenance:manage`. Queries: none.

Request (required: name):

```
{
  "name": "Monthly inspection",
  "categoryId": "{{categoryId}}",
  "triggerType": "TIME_BASED",
  "frequencyDays": 30,
  "tasks": [
    "Inspect cables",
    "Check operation"
  ],
  "description": "Monthly checks"
}
```

Success 201:

```
{
  "data": {
    "id": "example-id",
    "name": "Monthly inspection",
    "categoryId": "{{categoryId}}",
    "triggerType": "TIME_BASED",
```

```
{
  "frequencyDays": 30,
  "tasks": ["Inspect cables", "Check operation"],
  "description": "Monthly checks"
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

PUT /maintenance-templates/{id}

Permission: maintenance:manage. Queries: none.

Request (all fields optional):

```
{
  "name": "Monthly inspection",
  "categoryId": "{{categoryId}}",
  "triggerType": "TIME_BASED",
  "frequencyDays": 30,
  "tasks": [
    "Inspect cables",
    "Check operation"
  ],
  "description": "Monthly checks"
}
```

Success 200:

```
{
  "data": {
    "id": "example-id",
    "name": "Monthly inspection",
    "categoryId": "{{categoryId}}",
    "triggerType": "TIME_BASED",
    "frequencyDays": 30,
    "tasks": ["Inspect cables", "Check operation"],
    "description": "Monthly checks"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

DELETE /maintenance-templates/{id}

Permission: maintenance:manage. Queries: none.

Request body: none.

Success 200:

```
{
  "data": {
    "deleted": true
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /work-orders

Permission: **Authenticated** . Queries: page, limit, status, priority, assetId, assignedToUserId, scheduledFrom, scheduledTo.

Request body: none.

Success **200**:

```
{
  "data": [
    {
      "id": "example-id",
      "assetId": "{{assetId}}",
      "templateId": "{{templateId}}",
      "assignedToUserId": "{{userId}}",
      "title": "Inspect laptop",
      "description": "Check connections",
      "priority": "MEDIUM",
      "status": "OPEN",
      "dueDate": "2026-10-01T09:00:00Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "pages": 1
  }
}
```

Errors: **400**, **401**, **403**, **404**, **409**, **500** as defined above.

GET /work-orders/{id}

Permission: **Authenticated** . Queries: none.

Request body: none.

Success **200**:

```
{
  "data": {
    "id": "example-id",
    "assetId": "{{assetId}}",
    "templateId": "{{templateId}}",
    "assignedToUserId": "{{userId}}",
    "title": "Inspect laptop",
    "description": "Check connections",
    "priority": "MEDIUM",
    "status": "OPEN",
    "dueDate": "2026-10-01T09:00:00Z"
  }
}
```

Errors: **400**, **401**, **403**, **404**, **409**, **500** as defined above.

POST /work-orders

Permission: `workorder:manage` . Queries: none.

Request (required: `assetId`, `title`):

```
{
  "assetId": "{{assetId}}",
  "templateId": "{{templateId}}",
  "assignedToUserId": "{{userId}}",
  "title": "Inspect laptop",
  "description": "Check connections",
  "priority": "MEDIUM",
  "status": "OPEN",
  "dueDate": "2026-10-01T09:00:00Z"
}
```

Success `201`:

```
{
  "data": {
    "id": "example-id",
    "assetId": "{{assetId}}",
    "templateId": "{{templateId}}",
    "assignedToUserId": "{{userId}}",
    "title": "Inspect laptop",
    "description": "Check connections",
    "priority": "MEDIUM",
    "status": "OPEN",
    "dueDate": "2026-10-01T09:00:00Z"
  }
}
```

Errors: `400` , `401` , `403` , `404` , `409` , `500` as defined above.

PUT `/work-orders/{id}`

Permission: `workorder:manage` . Queries: none.

Request (all fields optional):

```
{
  "status": "IN_PROGRESS"
}
```

Success `200`:

```
{
  "data": {
    "id": "example-id",
    "assetId": "{{assetId}}",
    "templateId": "{{templateId}}",
    "assignedToUserId": "{{userId}}",
    "title": "Inspect laptop",
    "description": "Check connections",
    "priority": "MEDIUM",
    "status": "OPEN",
    "dueDate": "2026-10-01T09:00:00Z"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /locations/tree

Permission: Authenticated. Queries: none.

Request body: none.

Success 200:

```
{
  "data": []
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /assets/{id}/history

Permission: Authenticated. Queries: none.

Request body: none.

Success 200:

```
{
  "data": [
    {
      "type": "transfer",
      "action": "TRANSFERRED",
      "at": "2026-09-22T09:00:00Z",
      "data": {
        "assetId": "asset-id",
        "fromLocationId": "old-id",
        "toLocationId": "new-id"
      }
    }
  ]
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /warranties/expiring

Permission: Authenticated. Queries: page, limit, days.

Request body: none.

Success 200:

```
{
  "data": [
    {
      "id": "example-id",
      "assetId": "{{assetId}}",
      "provider": "Manufacturer",
      "startDate": "2026-01-01",
      "endDate": "2027-01-01",
      "terms": "Parts and labor"
    }
  ]
}
```

```

    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "pages": 1
  }
}

```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /work-orders/due

Permission: `Authenticated`. Queries: `page`, `limit`.

Request body: none.

Success 200:

```

{
  "data": [
    {
      "id": "example-id",
      "assetId": "{{assetId}}",
      "templateId": "{{templateId}}",
      "assignedToUserId": "{{userId}}",
      "title": "Inspect laptop",
      "description": "Check connections",
      "priority": "MEDIUM",
      "status": "OPEN",
      "dueDate": "2026-10-01T09:00:00Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "pages": 1
  }
}

```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

PUT /work-orders/{id}/complete

Permission: `workorder:manage`. Queries: none.

Request (all fields optional):

```

{
  "laborCost": 100,
  "partsCost": 50,
  "downtimeHours": 2,
  "outcome": "Fixed",
  "notes": "Replaced cable"
}

```

Success 200:

```
{
  "data": {
    "id": "work-order-id",
    "status": "COMPLETED",
    "completedAt": "2026-09-22T09:00:00Z",
    "serviceEvents": [
      {
        "workOrderId": "{{workOrderId}}",
        "laborCost": "100",
        "partsCost": "50",
        "downtimeHours": 2,
        "outcome": "Fixed",
        "notes": "Replaced cable",
        "cost": "150"
      }
    ]
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

GET /service-events

Permission: `Authenticated`. Queries: `page`, `limit`, `workOrderId`.

Request body: none.

Success 200:

```
{
  "data": [
    {
      "id": "example-id",
      "workOrderId": "{{workOrderId}}",
      "laborCost": "100",
      "partsCost": "50",
      "downtimeHours": 2,
      "outcome": "Fixed",
      "notes": "Replaced cable",
      "cost": "150"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "pages": 1
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.

POST /service-events

Permission: `workorder:manage`. Queries: none.

Request (required: `workOrderId`):

```
{
  "workOrderId": "{{workOrderId}}",
  "laborCost": 100,
  "partsCost": 50,
  "downtimeHours": 2,
  "outcome": "Fixed",
  "notes": "Replaced cable"
}
```

Success 201:

```
{
  "data": {
    "id": "example-id",
    "workOrderId": "{{workOrderId}}",
    "laborCost": "100",
    "partsCost": "50",
    "downtimeHours": 2,
    "outcome": "Fixed",
    "notes": "Replaced cable",
    "cost": "150"
  }
}
```

Errors: 400, 401, 403, 404, 409, 500 as defined above.